

逻辑回归

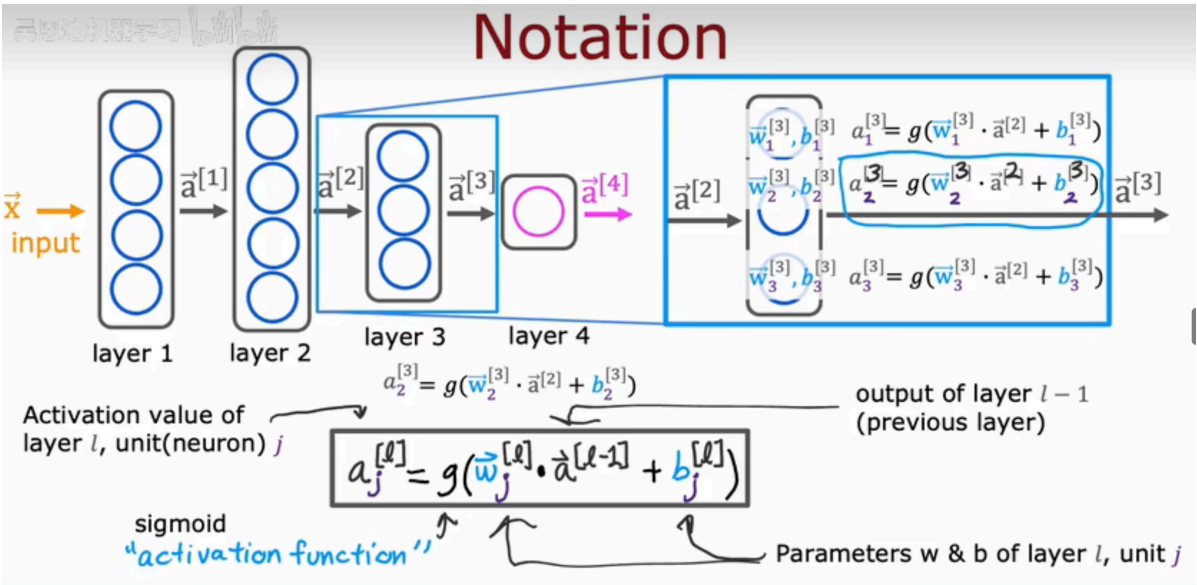
逻辑回归是概率型非线性回归，但其本质是线性回归，只是在特征到结果的映射中加入了一层函数映射，即先把特征线性求和，再使用 sigmoid 函数预测。

- 表达式 $p = \frac{1}{1+e^{-w^T x}}$ ，将p视为样本x作为正例的可能性， $\ln \frac{p}{1-p} = w^T x$ ，是在用线性回归模型的预测结果去逼近真实标记的对数几率，故逻辑回归又称对数几率回归
- 对称性： $\sigma(-a) = 1 - \sigma(a)$
 - 反转性： $a = \ln \frac{\sigma}{1-\sigma}$
 - 可导性： $\frac{d\sigma}{da} = \sigma(1 - \sigma)$
- 将后验概率表示为作用于变量x的线性函数中的 Logistic Sigmoid：
 $p(C_1|x) = y(x) = \sigma(w^T x), p(C_2|x) = 1 - p(C_1|x)$
- 似然函数 $p(t|w) = \prod_{i=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}, y_n = p(C_1|x_n)$
- 误差函数：交叉熵损失函数 $E(w) = -\ln p(t|w) = -\sum_{i=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$
- $\nabla E(w) = \sum_{i=1}^N (y_n - t_n) x_n = 0$

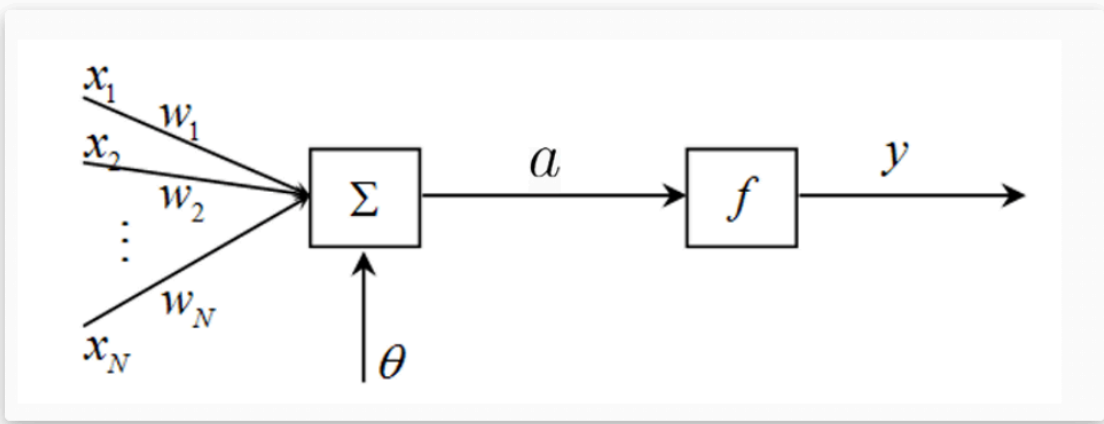
生成式模型和判别式模型

维度	生成式模型	判别式模型
定义 核心	先建模类条件密度 $p(x C_k)$ /联合分布 $p(x, C_k)$ ，再通过贝叶斯定理推导后验概率 $p(C_k x)$	直接对后验概率 $p(C_k x)$ 建模
优点	信息丰富；单类问题灵活性强；支持增量学习；可合成缺失数据	类间差异清晰；分类边界灵活；学习过程简单；分类性能较优
缺点	学习过程复杂；为匹配数据分布，可能牺牲分类性能	无法反映数据特性；需全量数据训练，不支持增量学习
相互 关系	生成式模型可推导得到判别式模型	判别式模型无法反推出生成式模型

反向传播算法



首先记住这个神经元的基本结构，尤其记住各个符号的含义：



神经元的基本结构

1. $x_1 \cdots x_N$: 前一层的输入
2. $w_1 \cdots w_N$: 权值，也是神经网络训练的目标
3. Σ : 求和器
4. θ : 求和器阈值，可有可无
5. a :

$$a = \sum_{i=1}^N x_i w_i - \theta$$

6. f : 激活函数，就是ReLU啊、Sigmoid啊之类的
7. y : 神经元输出

$$y = f(a)$$

之后的神经网络图中，每一个「圆点」，其实都暗含了这些东西。

所谓的反向传播算法，就是定义一个损失函数：

$$E(w) = \frac{1}{2} \sum_{i=1}^N (y(x_i, w) - t_i)^2$$

计算它的梯度 $\nabla E(w)$ ，然后用梯度下降法更新 w 。

算法可以分为两个阶段：

1. 前馈(正向过程)：从输入层经隐层逐层正向计算各单元的输出；
2. 学习(反向过程)：由输出误差逐层反向计算隐层各单元的误差，并用此误差修正前层的权值

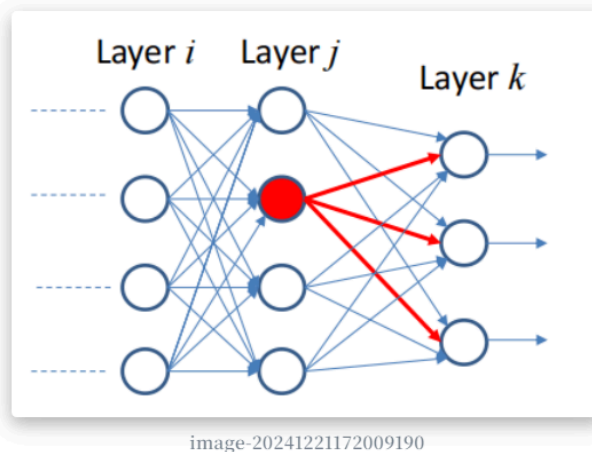


image-20241221172009190

现在，我们要计算 E 对 w 的导数，记激活函数为 h 。我们知道 E 是 y 的函数， y 是 a 的函数， a 是 w, x 的函数，故：

$$\begin{aligned}\frac{\partial E_n}{\partial w_{kj}} &= \frac{\partial E_n}{\partial y_k} \frac{\partial y_k}{\partial a_k} \frac{\partial a_k}{\partial w_{kj}} \\ &= (y_k - t_k) \frac{\partial y_k}{\partial a_k} \frac{\partial a_k}{\partial w_{kj}} \\ &= (y_k - t_k) h'(a_k) \frac{\partial a_k}{\partial w_{kj}} \\ &= (y_k - t_k) h'(a_k) x_j\end{aligned}$$

现在，如果要计算 j 层的梯度，即：

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}}$$

接下来使用两个记号：

$$\delta_j = \frac{\partial E_n}{\partial y_j} \frac{\partial y_j}{\partial a_j} = \frac{\partial E_n}{\partial a_j}$$

和

$$z_i = \frac{\partial a_k}{\partial w_{ki}}$$

则有

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$$

因为上面已经求出了输出层的误差，根据误差反向传播的原理，当前层的误差可理解为上一层所有神经元误差的复合函数，即使用上一层的误差来表示当前层误差，并依次递推。有：

$$\delta_j = \frac{\partial E_n}{\partial a_j} = \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}$$

其中

$$a_k = \sum_k w_{kj} h(a_j)$$

故：

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

这样，我们就推导出了反向传播的递推式。

整体算法流程

1. 初始化权重 w_{ij}
2. 对于输入的训练样本，求取每个节点输出和最终输出层的输出值
3. 对于输出层求

$$\delta_k = y_k - t_k$$

4. 对于隐藏层求

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

5. 求输出误差对于每个权重的梯度

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j x_i$$

6. 更新权重

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E(\mathbf{w}^{(\tau)})$$

此外，还要记住一个结论：

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

其中 σ 是Sigmoid函数，即

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

其他神经网络

- 径向基函数神经网络

只有一个隐层，隐层单元采用径向基函数作为其输出函数，输入层到隐层之间的权值均固定为1；输出节点为线性求和单元，隐层到输出节点之间的权值可调，因此，输出为隐层的加权求和

- Hopfield网络

是一种反馈网络。反馈网络具有一般非线性系统的许多性质，如稳定性问题等，在某些情况下还有随机性、不可预测性。因此它比前馈网络的内容复杂，具有权值对称、无自反馈的特点

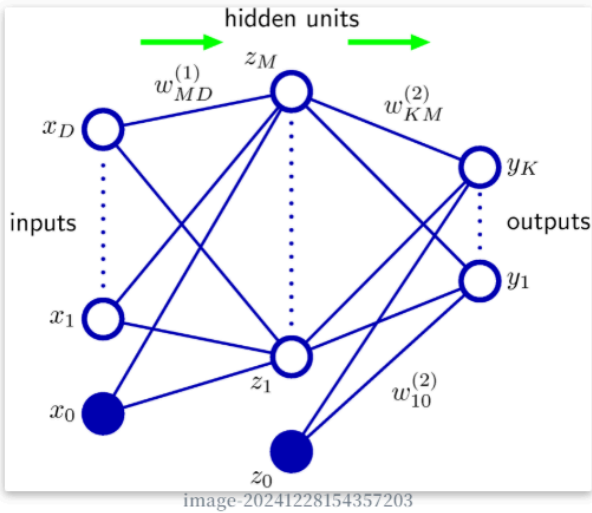
- 自适应共振理论神经网络

通过反复将输入学习模式由输入向输出层自下而上短时记忆和由输出向输入层自上而下长期记忆和比较来实现。当记忆和比较达到共振时，输出矢量可正确反映输入学习模式的分类，且网络原有记忆不受影响

- 能将任意维输入模式在输出层映射成一或二维离散图形，并保持拓扑结构不变
- 在输出层，获胜神经元邻域内的神经元在不同程度上都得到兴奋，而在邻域外的神经元都被抑制
- 邻域可为任意形状，但一般是对称的。邻域是时间的函数，随时间增大而减小，最后可能剩下一个或一组神经元。最终得到的区域反映了一类输入模式的属性

例题1

【例题1】 包含一层隐藏层的前馈神经网络如图所示，其中给定训练集 $D = \{(x_1, t_1), (x_2, t_2), \dots, (x_N, t_N)\}$, $x_i \in \mathbb{R}^D, t_i \in \mathbb{R}^K$ 。隐藏层激活函数为 $h(x)$, 输出层激活函数为 $\sigma(x)$ 。 y_n 表示第 n 个样本 x_n 对应的神经网络输出向量, $y_n = [y_{n1}, \dots, y_{nk}, \dots, y_{nK}]^T$, 准则函数为 $E_n(w) = \frac{1}{2} \sum_{k=1}^K \{y_{nk} - t_{nk}\}^2$ 。网络包含 D 个输入神经元, K 个输出神经元, 以及 M 个隐层神经元。试推导反向传播算法中对每一层权值参数 ($\omega_{md}^{(1)}$ 与 $\omega_{km}^{(2)}$) 的更新



【解答1】 先推导前向传播：

$$\begin{aligned} a_m &= \sum_{d=1}^D x_d w_{md}^{(1)} \\ z_m &= h(a_m) \\ a_k &= \sum_{m=0}^M z_m w_{km}^{(2)} \\ y_{nk} &= \sigma(a_k) \end{aligned}$$

对输出层：

$$\begin{aligned} \delta_k &= \frac{\partial E_n}{\partial a_k} \\ &= \frac{\partial E_n}{\partial y_{nk}} \frac{\partial y_{nk}}{\partial a_k} \\ &= (y_{nk} - t_{nk}) \sigma'(a_k) \end{aligned}$$

对隐藏层：

$$\begin{aligned}\delta_m &= \frac{\partial E_n}{\partial a_m} \\ &= \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_m}\end{aligned}$$

其中,

$$a_k = \sum h(a_m)w_{km}^{(2)}$$

则有：

$$\begin{aligned}\delta_m &= \frac{\partial E_n}{\partial a_m} \\ &= \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_m} \\ &= h'(a_m) \sum_k \delta_k w_{km}^{(2)}\end{aligned}$$

则权值更新公式为：

$$\begin{aligned}\frac{\partial E_n}{\partial \omega_{km}^{(2)}} &= \delta_k \cdot \frac{\partial a_k}{\partial \omega_{km}^{(2)}} = z_m \cdot (y_{nk} - t_{nk}) \cdot \sigma'(a_k) \\ \frac{\partial E_n}{\partial \omega_{md}^{(1)}} &= \delta_m \cdot \frac{\partial a_m}{\partial \omega_{md}^{(1)}} = x_d \cdot h'(a_m) \sum_k \delta_k \cdot \omega_{km}^{(2)}\end{aligned}$$

例题2

【例题2】考虑只有一个神经元的多层神经网络, 其中 $x_{i+1} = \sigma(z_i) = \sigma(w_i x_i + b_i) (i \in [1, 4])$, $C = \text{Loss}(x_5)$, σ 表示 sigmoid 函数 $f(x) = 1/(1 + e^{-x})$, 且 $|w_i| < 1$ 。假设已知 $\frac{\partial C}{\partial b_5}$, 推导 $\frac{\partial C}{\partial b_i} (i \in [1, 4])$, 并阐述当神经网络层数过深时, 梯度消失的原因。

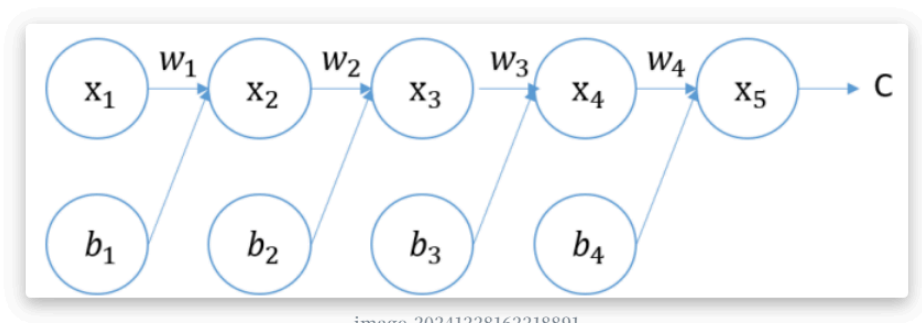


image-20241228162218891

【解答2】

1. $i = 4$ 时, 有: $x_5 = \sigma(z_5), z_5 = w_4 x_4 + b_4$, 因此:

$$\frac{\partial C}{\partial b_4} = \frac{\partial C}{\partial x_5} \frac{\partial x_5}{\partial z_4} \frac{\partial z_4}{\partial b_4} = \frac{\partial C}{\partial x_5} \sigma'(z_4)$$

2. $i = 3$ 时, 有: $x_4 = \sigma(z_4), z_4 = w_3 x_3 + b_3$, 则:

$$\frac{\partial C}{\partial b_3} = \frac{\partial C}{\partial x_5} \frac{\partial x_5}{\partial z_4} \frac{\partial z_4}{\partial x_4} \frac{\partial x_4}{\partial z_3} \frac{\partial z_3}{\partial b_3} = \frac{\partial C}{\partial x_5} \sigma'(z_4) w_4 \sigma'(z_3)$$

3. $i = 2$ 时, 有: $x_3 = \sigma(z_3), z_3 = w_2 x_2 + b_2$, 则:

$$\frac{\partial C}{\partial b_2} = \frac{\partial C}{\partial x_5} \frac{\partial x_5}{\partial z_4} \frac{\partial z_4}{\partial x_4} \frac{\partial x_4}{\partial z_3} \frac{\partial z_3}{\partial x_3} \frac{\partial x_3}{\partial z_2} \frac{\partial z_2}{\partial b_2} = \frac{\partial C}{\partial x_5} \sigma'(z_4) w_4 \sigma'(z_3) w_3 \sigma'(z_2)$$

4. $i = 1$ 时, 同理, 有:

$$\frac{\partial C}{\partial b_1} = \frac{\partial C}{\partial x_5} \sigma'(z_4) w_4 \sigma'(z_3) w_3 \sigma'(z_2) w_2 \sigma'(z_1)$$

由于 $\sigma'(z) \leq \frac{1}{4}$ 且 $|w| < 1$, 则当层数过深时, $\frac{\partial C}{\partial b_i}$ 趋于 0, 则梯度消失